



The Shell

NCSC₁₀₁

Prof. Pranay Kumar Saha

July 29, 2025



What is the Shell?

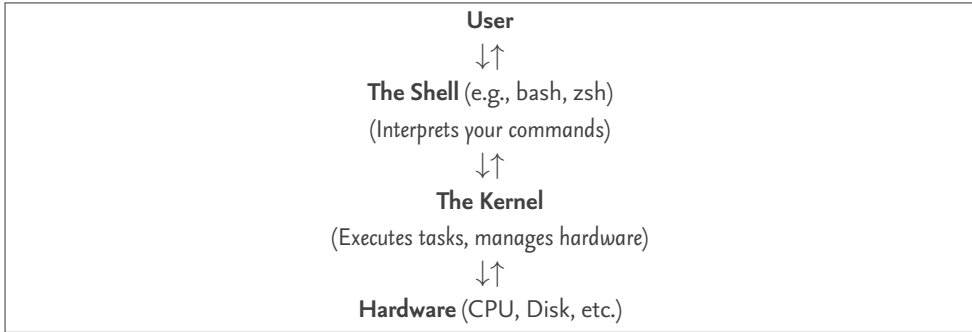
The Command Interpreter

The Shell is an essential program that acts as the primary interface between a user and the operating system's kernel. It is a **command-line interpreter (CLI)**.

- When you type a command, the shell reads it, interprets it, and asks the kernel to perform the requested task.
- It's a powerful tool for controlling and automating your system.
- It is both an interactive command language and a scripting language.



The Shell's Role



The shell is the translator that allows you, the user, to communicate your instructions to the core of the operating system.



Understanding the Command Prompt

The prompt is displayed by the shell to indicate it's ready for your command. It often contains useful information.

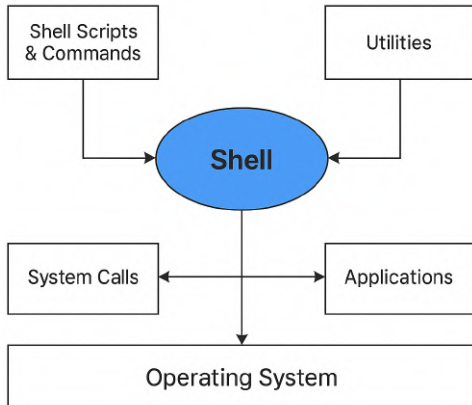
A common prompt format:

```
user@hostname: /directory$
```

- user: Your current username.
- hostname: The name of the computer you are working on.
- ~/directory: The current working directory (~ is a shortcut for your home directory).
- \$: The prompt symbol. For a regular user, it's typically \$. For the superuser (root), it's #.



Shell & Command Prompt



```
userlocalhost:$ ls
Documents Downloads
userlocalhost:$
cd Documents
userlocalhost:Documents
$
```



Commands vs. Utilities: What's the Difference?

Often Used Interchangeably

In everyday language, we call everything we type a "command." However, there's a technical distinction.

- **Utility (or Tool):** An external program that performs a task (e.g., `ls`, `grep`). The shell finds this program on the disk and runs it.



Commands vs. Utilities: What's the Difference?

Often Used Interchangeably

In everyday language, we call everything we type a "command." However, there's a technical distinction.

- **Utility (or Tool):** An external program that performs a task (e.g., `ls`, `grep`). The shell finds this program on the disk and runs it.
- **Shell Built-in:** A command that is part of the shell itself (e.g., `cd`, `pwd`, `echo`). These are faster because no external program needs to be loaded.

Key Idea

Every instruction you give is a **command**, but that command might be executing an external **utility** or a **shell built-in**.



Types of Shells

There are many different shells, each with its own features.

Shell	Description
sh	The original Bourne Shell. Simple and foundational.
bash	B ourne- A gain S hell. The most common default shell.
csh	The C Shell, with a C-like syntax.
ksh	The Korn Shell, backward-compatible with sh.
zsh	The Z Shell. Highly customizable with many features.
fish	F riendly I nteractive S hell. Focuses on usability.

Our examples will primarily use bash/zsh syntax, which is widely applicable.



Navigating the Filesystem

These commands help you move around and know where you are.

pwd - Print Working Directory

Tells you the full path of your current location.

```
$ pwd
/home/user/documents
```

cd - Change Directory

Moves you to a different directory.

```
$ cd /var/log
$ cd ..    # Go up one level
$ cd      # Go to your home directory
```



Viewing Files and Directories

ls - List Directory Contents

Shows the files and directories in the current location.

```
$ ls  
file1.txt image.jpg project_folder
```



Viewing Files and Directories (Contd..)

ls with Options (Flags)

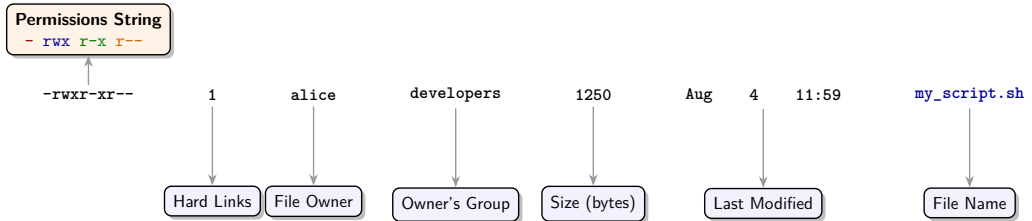
Flags modify a command's behavior.

- `-l`: Long listing format (shows permissions, owner, size, etc.).
- `-a`: Shows all files, including hidden ones (starting with `.`).

```
$ ls -la
drwxr-xr-x 2 user user 4096 Jul 29 11:30 .
-rw-r--r-- 1 user user 134 Jul 29 11:30 .bashrc
-rw-r--r-- 1 user user 45 Jul 29 11:25 file1.txt
```



Understanding `ls -al` output





Understanding File Permissions

The output of `ls -l` contains crucial security information.

Decoding the permission string: `-rwxr-xr--`

- `-rwxr-xr--` : **File Type**. Common types include:
 - `-` : Regular File - need to store user data
 - `d` : Directory - need to organize the file-system
 - `l` : Symbolic Link - need for shortcuts
 - `c` : Character Device - need to interact with stream-based hardware such as keyboards, mice, and serial ports
 - `b` : Block Device -need to interact with storage hardware

```
$ ls -l my_script.sh
-rwxr-xr-- 1 alice developers 1250 Aug 4 10:43 my_script.sh
```



Understanding File Permissions

The output of `ls -l` contains crucial security information.

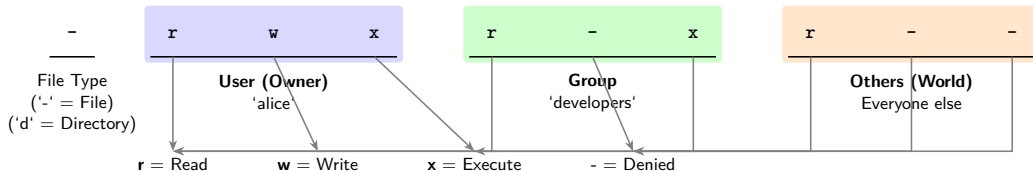
Decoding the permission string: `-rwxr-xr--`

- `-rwxr-xr--` : **File Type**. `-` for a file, `d` for a directory.
- `-rwxr-xr--` : **User (Owner)** permissions. The owner can read, write, and execute.
- `-rwxg-xr--` : **Group** permissions. Members of the group can read and execute, but not write.
- `-rwxr-xo--` : **Others** permissions. Everyone else can only read the file.

```
$ ls -l my_script.sh
-rwxr-xr-- 1 alice developers 1250 Jul 29 12:00 my_script.sh
```



Understanding File Permissions





The Meaning of Read, Write, Execute

The permissions have different meanings for files and directories.

For a FILE

- **Read (r):** Can view the contents of the file.
- **Write (w):** Can modify or delete the contents of the file.
- **Execute (x):** Can run the file as a program or script.

For a DIRECTORY

- **Read (r):** Can list the contents of the directory (i.e., use `ls`).
- **Write (w):** Can create, rename, or delete files within the directory.
- **Execute (x):** Can enter the directory (i.e., use `cd`).



Questions

```
$ ls -l my_script.sh
-rwxr-xr-- 1 alice developers 1250 Aug 4 10:43 my_script.sh
```

Questions for you

- Based on the example, what can members of the ‘developers’ group do with this file?
- Can a user who is not ‘alice’ and not in the ‘developers’ group modify this script? Why or why not?



Changing Permissions: `chmod` (Symbolic)

The symbolic method is intuitive and easy to read.

Syntax: `chmod [who][operator][permission] file`

- **Who:** u (user), g (group), o (others), a (all).
- **Operator:** + (add), - (remove), = (set exactly).
- **Permission:** r (read), w (write), x (execute).



Changing Permissions: `chmod` (Symbolic)(contd...)

Examples

```
# Add execute permission for the user (owner)
```

```
$ chmod u+x backup.sh
```

```
# Remove write permission for the group
```

```
$ chmod g-w config.yml
```

```
# Set permissions for others to read-only
```

```
$ chmod o=r important.dat
```



Changing Permissions: `chmod` (Octal)

The octal (numeric) method is faster for setting all permissions at once.

Numeric Values

$r = 4, \quad w = 2, \quad x = 1$

Permissions for each class (user, group, other) are the sum of their values.

- $7 = 4+2+1 = rwx$
- $6 = 4+2 = rw-$
- $5 = 4+1 = r-x$
- $4 = 4 = r--$



Changing Permissions: `chmod` (Octal)(contd...)

Examples

```
# rwxr-xr-x (user=7, group=5, others=5)
$ chmod 755 my_script.sh

# rw-r--r-- (user=6, group=4, others=4)
$ chmod 644 my_document.txt
```



Managing Files and Directories

mkdir - Make Directory

```
$ mkdir new_project
```

cp - Copy

```
# Copy file  
$ cp source.txt dest.txt  
  
# Copy directory  
$ cp -r project/ backup/
```

mv - Move / Rename

```
# Rename  
$ mv old.txt new.txt  
  
# Move  
$ mv report.pdf docs/
```



Managing Files and Directories(contd..)

rm - Remove

```
# Remove file  
$ rm temp.log  
  
# Remove directory  
$ rm -r old_project
```

Warning!

rm is permanent! There is no "Recycle Bin" on the command line. Use with caution.



Viewing File Contents

cat - Concatenate and Display

Prints the entire content of a file to the screen.

```
$ cat config.txt
```

less - View File Page by Page

Lets you scroll through large files. More powerful than more.

```
$ less /var/log/syslog
```

(Press q to quit, arrow keys to navigate).



Viewing File Contents(contd..)

head / tail

View the beginning or end of a file.

```
$ head -n 5 file.txt # First 5 lines  
$ tail -n 10 file.txt # Last 10 lines
```



Getting Help

man - The Manual

The most important command! It shows you the manual page for any other command.

```
$ man ls
```

This will open a detailed document explaining what `ls` does and all its available options.

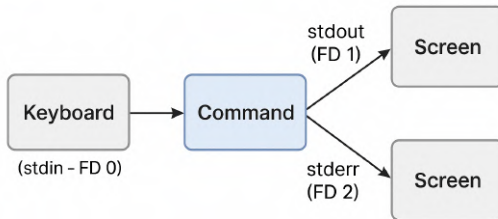
- Navigate man pages just like `less`.
- Press `q` to exit.
- When in doubt, man it out!



Standard Streams: stdin, stdout, stderr

Every command has three standard data streams associated with it.

- **Standard Input (stdin):** Where the command gets its input from. By default, this is your keyboard. (File Descriptor 0)
- **Standard Output (stdout):** Where the command sends its normal output. By default, this is your screen. (File Descriptor 1)
- **Standard Error (stderr):** Where the command sends its error messages. By default, this is also your screen. (File Descriptor 2)





I/O Redirection: Changing the Flow

You can redirect these streams to or from files.

Redirecting stdout (>)

Sends normal output to a file instead of the screen. **Overwrites the file.**

```
$ ls -l > file_list.txt
```

Appending stdout (>>)

Sends normal output to a file. **Appends to the end of the file.**

```
$ echo "New log entry" >> application.log
```



I/O Redirection: Changing the Flow

Redirecting stdin (<)

Takes input from a file instead of the keyboard.

```
$ sort < names.txt
```



Pipes: Chaining Commands

The Pipe Operator (|)

A pipe is one of the most powerful features. It sends the **stdout** of the command on the left to the **stdin** of the command on the right.

Command A (stdout) -> (stdin) Command B

Example: Find all **.txt** files

- `ls -l`: Produces a long list of all files.
- `grep ".txt"`: Searches its input for lines containing ".txt".

```
$ ls -l | grep ".txt"
-rw-r--r-- 1 user user 45 Jul 29 11:25 file1.txt
-rw-r--r-- 1 user user 89 Jul 29 11:40 notes.txt
```



The Shell

Thank You for Listening!